



ECOLE
POLYTECHNIQUE
DE BRUXELLES

2025-2026

INFO H 502 – 3D GRAPHICS

Implementation of a 3D scene

Mandel Eliot 493519

Bonatto Daniele

Yildirim Emirhan 459800

Soetens Elise

25 mai 2026

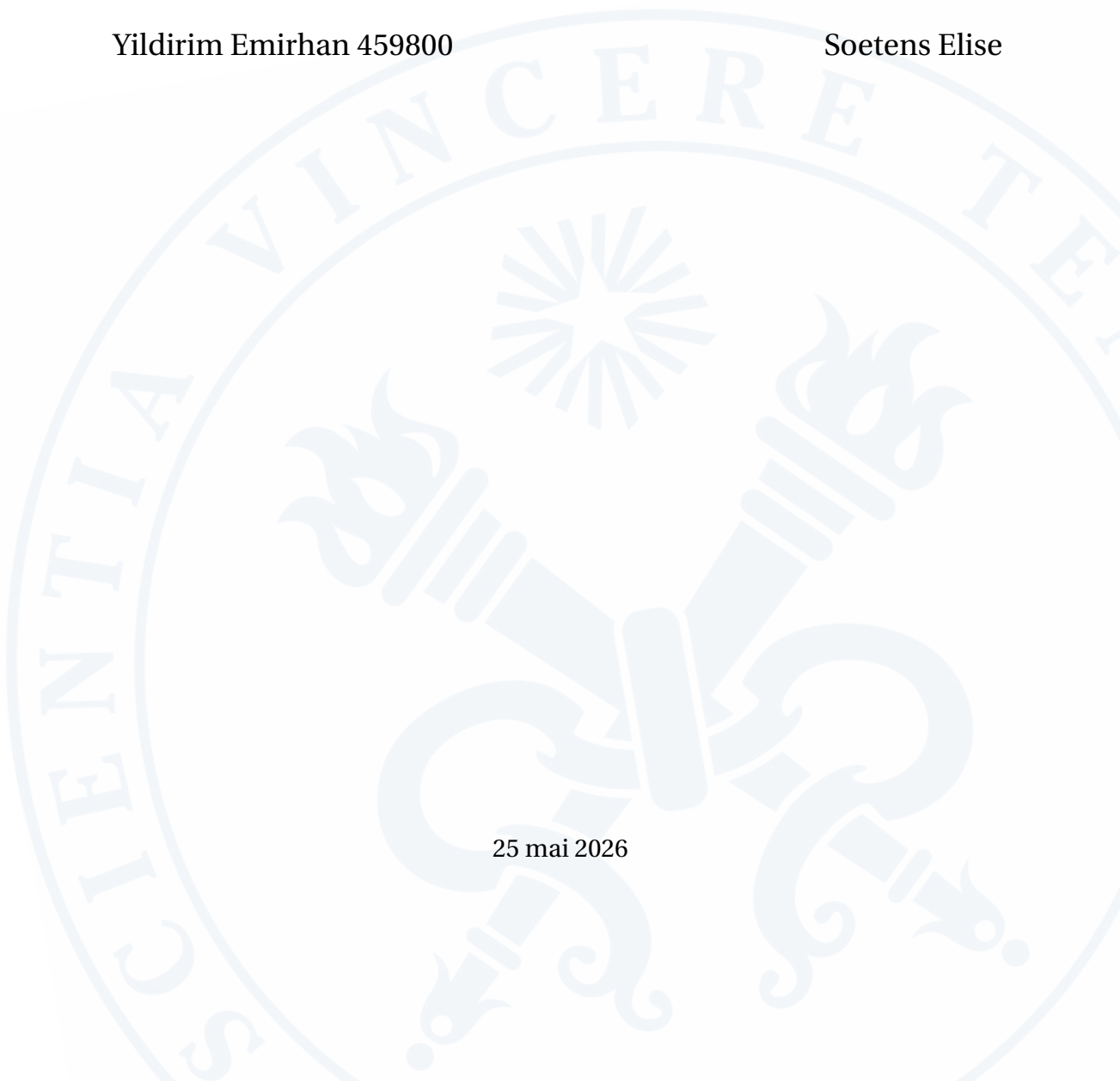


Table des matières

1	Basic Features	1
1.1	Lights	1
1.2	Textures	1
1.3	Loading Models	2
1.4	Cubemap	2
1.5	Game Logic	2
1.6	Object Moving in the Scene	3
1.7	Free Navigation	3
1.8	Reflection / Refraction	3
2	Intermediate Features	4
3	Advanced Features	5
3.1	Shadow Map	5
3.2	Percentage-Closer Filtering	6

The project is available on GitHub : https://github.com/MandRepo/INF0H502_Mandel_Yildirim_Project.git

1.1 Lights

Lights are implemented in several ways. First, a soft ambient light is applied to the entire scene ; from a realistic standpoint, this simulates the setting sun depicted in the cubemap texture. Next, each streetlight generates a light source of a different color.

This light appears as diffuse light on the asphalt, the grass, the car, the rabbit, and the wall. It is also attenuated quadratically to simulate localized lighting, thereby preventing a streetlight on one side of the map from illuminating the other side. Finally, only the wall receives specular lighting (Phong model). When this is applied to other objects, the specular component is projected onto the object's normal in the direction between the camera and the light source, which produces an unintuitive result : the light is not visible when the car is directly under the streetlight, but only when it is slightly set back and at an equal distance between the camera and the light source. In order to maintain a realistic rendering while illustrating this effect, we chose to implement it only for the wall.

1.2 Textures

Multiple textures are loaded :

- the grass on the ground.
- the asphalt on the race track.
- the texture of the streetlights.

- the cubemap.
- the bricks on the wall. These come from a 2D photo and are therefore implemented using the same logic as a cubemap, but this time with outward rendering.

1.3 Loading Models

Several objects are loaded into the simulation. These are loaded using `object.h`, which reads the various objects and sends the relevant information to `shader.h`, which processes the received data. `shader.h` is responsible for storing the various object elements in the VAOs and VBOs so that the GPU can process the relevant data.

Models are loaded for the following objects :

- the walls.
- a ground plane for the grass.
- the race track (created in Blender).
- the streetlights (downloaded from the Internet¹, along with their respective textures).
- the car (taken from the Internet² in the *smooth* version to limit the amount of computation and maintain a certain fluidity in our program, and without a texture);
- the rabbit, which is implemented with the same shader as the car to better illustrate reflection and refraction given its more complex geometry combined with its fixed position.
- the cube for the cubemap.

1.4 Cubemap

The cubemap is loaded from an image found online depicting a nighttime landscape, to capture the atmosphere at nightfall. In addition, we added a few modifications (points of light) representing stars to enhance the contrast in the reflection.

1.5 Game Logic

Our scene depicts a car race. Although the game mechanics are not yet fully implemented, the goal is to complete as many laps as possible in the fastest time. The car emits smoke when it drifts,

1. <https://free3d.com/fr/3d-model/light-pole-32057.html>

2. <https://free3d.com/fr/3d-model/low-poly-car--49763.html>

which occurs during turns. It can accelerate, decelerate, and drift (turn). This acceleration (deceleration) is progressive, and the car gradually decelerates when no controls are active to simulate real-world behavior.

1.6 Object Moving in the Scene

The car can be moved using the following keys :

- **W** : accelerate forward;
- **S** : brake / reverse;
- **A** : turn left (drift);
- **D** : turn right (drift).

1.7 Free Navigation

Pause mode, activated using the **P** key, allows the camera to detach from the vehicle and navigate freely within the scene space. The **W**, **A**, **S**, and **D** keys control movement and thus position, while the arrow keys control the camera's orientation.

1.8 Reflection / Refraction

The car reflects its surroundings via the cubemap. Pressing **R** switches it to refraction. This allows you to simulate both behaviors at will. The rabbit in the middle of the map is linked to the car's shaders. The goal is simply to represent this behavior more visually given the rabbit's geometry. Its behavior also changes when the 'R' key is pressed, simultaneously with the car.

Intermediate Features

Particles have been implemented to simulate the car's drift. They are generated at the rear of the vehicle. Their speed, which is intentionally reduced to prevent them from drifting too far away, is directed in the direction of the drift : if the car turns left, the rear drifts to the right, causing the particles to drift to the right as well. This is intended to reproduce realistic behavior. Furthermore, since the car is moving at a certain speed, we ensure that when particles are generated, they are distributed uniformly between positions $t - 1$ and t .

3.1 Shadow Map

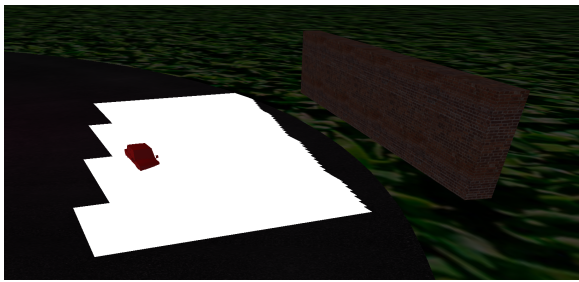
The *shadow map* shading technique was implemented to simulate the shadows cast by the directional light representing the moon. The technique involves two rendering passes. During the first pass, a virtual camera is positioned on the side of the light source and records, for each pixel, the depth of the nearest object. The result is stored in a *shadow map*, a depth texture. During the second pass, the main camera checks, for each rendered pixel, whether it is visible from the light camera by comparing its depth to the value stored in the shadow map. If the depth is greater, the pixel is in shadow and its intensity is reduced. This technique relies on the use of a *Framebuffer Object* (FBO).

Unlike a *Vertex Buffer Object* (VBO), which stores geometric data (positions, normals, texture coordinates) on the GPU, an FBO is an off-screen rendering target : it allows rendering to be re-directed to a texture rather than to the screen. It is in this FBO that the shadow map is generated during the first pass, before being passed to the fragment shader as a standard texture. To avoid the self-shading artifact known as *shadow acne* (a surface that casts a shadow on itself due to numerical inaccuracies), a depth bias is applied during the comparison. This bias slightly shifts the compared depth value.

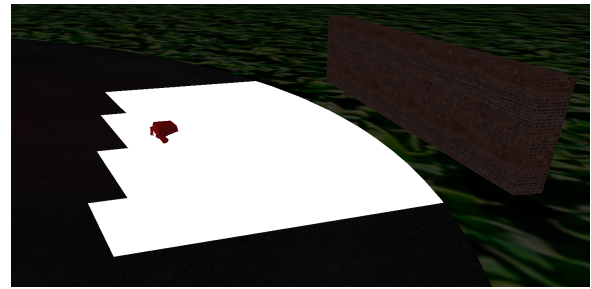
Calibrating it is tricky : too little bias reveals shadow acne, while too much bias can cause shadows to float or distort their shape (*peter panning*).

As can be seen in the images 3.1, if the offset is too small, the shadow is not generated all the way to the edge of the track, which does not reflect reality.

Furthermore, this model is sensitive to the surface's angle relative to the light source : the uns-moothed pattern visible on the left side of the image illustrates this artifact, caused by the fact that surfaces parallel to the light receive less accurate depth values, amplifying the *shadow acne*



(a) A shadow on the asphalt and the car with a bias of 0.001



(b) OA shadow on the asphalt and the car with a bias of 0.0001

FIGURE 3.1 – Comparison of the Effect of Depth Bias on Shadow Acne

even for moderate offsets. For simplicity's sake, only the wall casts a shadow on the car and on the asphalt. This logic could be extended to the rest of the environment, at the cost of additional computational overhead proportional to the number of occluding objects. Finally, a point light source placed at the center of the scene would have required a set of shadow maps for each direction (such as a cubemap) (six cameras covering each direction), making the implementation significantly more complex. This is why we chose to use a directional light source positioned at the edge of the scene, which allows us to limit ourselves to a single light camera and thus to a single FBO.

3.2 Percentage-Closer Filtering

Percentage-Closer Filtering (PCF) is a technique used to smooth the edges of shadows produced by a shadow map. As illustrated in Figure 3.1, the artifacts observed on grazing surfaces (i.e., surfaces whose inclination is steep relative to the direction of the light source) result in extremely pixelated and unsmoothed shadow edges. PCF provides a natural solution to this problem. The principle relies on multiple sampling of the shadow map. Rather than comparing the depth of the current pixel to a single texel in the shadow map, PCF examines a neighborhood of texels around the projected position, then calculates the average of the resulting binary values. The resulting shadow value is thus a real number between 0 and 1, producing a smooth transition between lit and shadowed areas. In the context of this project, this could have been implemented by modifying only the relevant fragment shaders (`car.frag` and `ground.frag`). The comparison determining whether or not to apply the shadow should be replaced by an average calculated over the neighboring texels of the shadow map. It is important to note that this approach differs fundamentally from the color gradient covered in the course : in the latter case, the color varies based on the pixel's UV coordinates, independently for each fragment. Here, on the contrary, a pixel's result depends explicitly on its neighbors in the shadow map, which justifies the additional complexity of

this technique.